

Reinforcement Learning

COMPLETE TUTORIAL: MAB, Monte Carlo, TD, DQN, Policy Methods

GARP RAI Exam Preparation

What You Will Learn in This Tutorial:

Topic	Key Concepts Covered
1. RL Fundamentals	Agent, Environment, State, Action, Reward, Policy, Value Functions
2. Multi-Armed Bandit	Exploration vs Exploitation, ϵ -Greedy, Q-Value Updates
3. Monte Carlo Methods	Episode-based Learning, Full Returns, Bias-Variance Trade-off
4. Temporal Difference	Online Learning, Bootstrapping, Q-Learning, SARSA, TD Error
5. Deep RL & DQN	Neural Networks, Experience Replay, Target Networks
6. Policy Methods	Value-based vs Policy-based, Actor-Critic, Continuous Actions

Tutorial Format:

- Each topic presented with [theory](#), [mathematics](#), [examples](#), and [exam traps](#)
- Step-by-step explanations with detailed calculations
- Practical tips for risk management applications

Key to Success: Understand the "[exploration vs exploitation](#)" trade-off - it is the heart of RL!

RL - What Makes It Different?

Reinforcement Learning vs Other Paradigms

Paradigm	Key Characteristic
Supervised Learning	Learns from labeled data ; output = prediction/classification
Unsupervised Learning	Finds hidden patterns ; output = clusters/structures
Reinforcement Learning	Learns through trial and error ; output = policy/action

Core RL Concept:

- Agent **interacts** with environment
- Takes **actions**
- Receives **rewards** (positive or negative)
- Goal: **Maximize cumulative future reward**

Analogy - Dog Training:

- Command → Dog takes action → Reward if correct
- Over time, dog learns the **policy**: command → correct action

Exam Trap: RL is NOT supervised learning - it uses **rewards** as feedback, not labels!

The Language of Reinforcement Learning

Term	Definition
Agent	The learner/decision-maker who takes actions
Environment	The world the agent interacts with
Action (a)	A choice the agent can make
State (s)	The current situation of the environment
Reward (R)	Feedback from environment (positive or negative)
Policy (π)	The agent's strategy - maps states to actions
Value Function $V(s)$	How good it is to be in state s
Action-Value $Q(s,a)$	How good it is to take action a in state s

Key Insight:

- The goal is to find the **optimal policy** π^* that maximizes reward
- If you know $Q(s,a)$ for all actions, the optimal policy is: $\pi^*(s) = \arg \max_a Q(s, a)$

Exam Trap: Policy $\neq$ Value Function. Policy is the **strategy**; Value Function is the **estimate of goodness**!

The Mathematics of Reinforcement Learning

Expected Future Return (G):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where γ is the **discount factor** ($0 \leq \gamma \leq 1$)

State-Value Function V(s):

$$V_\pi(s) = E_\pi[G_t | S_t = s]$$

Measures how good it is to be in state s under policy π

Action-Value Function Q(s,a):

$$Q_\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$$

Measures how good it is to take action a in state s

Relationship Between V and Q:

$$V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s, a)$$

$$V^*(s) = \max_a Q^*(s, a)$$

Exam Trap: $V(s)$ is the **expectation over actions**; $Q(s,a)$ is **action-specific**!

Real-World RL Applications

AlphaGo / AlphaZero (DeepMind):

- Learned to play Go, Chess, Shogi at [superhuman level](#)
- Played [millions](#) of games against itself
- Discovered [novel strategies](#) humans hadn't found

Robotics:

- Training robots for [complex manipulation](#)
- Self-driving cars
- Warehouse automation

Business Applications:

- [Dynamic Pricing](#): Adjust prices based on demand
- [Supply Chain](#): Inventory and routing optimization
- [Trading](#): Optimal execution of trades
- [Chatbots](#): RLHF for ChatGPT and other LLMs

RLHF - Reinforcement Learning from Human Feedback:

- Humans rank model outputs
- RL algorithm adjusts to [maximize human scores](#)
- Key to modern LLM capabilities

Exam Trap: RLHF is a key technique for [fine-tuning LLMs](#) - not just for games!

The Multi-Armed Bandit Problem

The Scenario:

- A gambler (agent) faces **multiple slot machines**
- Each machine has **unknown** reward distribution
- Goal: **Maximize total winnings** over many plays

Key Characteristics of MAB:

- 1 **Single state** - environment doesn't change
- 2 Actions don't affect future states
- 3 Each action gives an **immediate reward**
- 4 No long-term consequences of actions

Why MAB Matters:

- **Foundation** for understanding RL
- Introduces the **exploration vs exploitation** dilemma
- Real-world applications: A/B testing, ad optimization

The Core Dilemma:

- **Exploit**: Play the machine that has been best so far
- **Explore**: Try other machines to find if they are better
- Neither pure exploration nor pure exploitation is optimal

Exam Trap: MAB has **ONLY ONE STATE** - this is what makes it simpler than general RL!

MAB - The Three Strategies

Three Strategies for MAB

Strategy 1: Greedy (Pure Exploitation)

- Always choose the machine with the **highest Q-value**
- Pros: Maximizes immediate reward
- Cons: May miss a better machine that was never tried

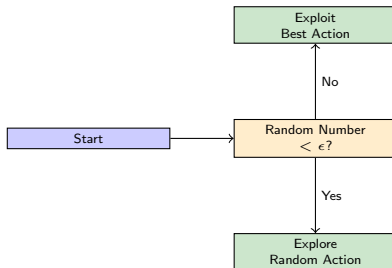
Strategy 2: Random (Pure Exploration)

- Always choose a **random** machine
- Pros: Gathers information about all machines
- Cons: Never uses information to maximize reward

Strategy 3: ϵ -Greedy (Balanced)

- With probability ϵ : **Explore** (random action)
- With probability $1 - \epsilon$: **Exploit** (best action so far)
- Pros: **Balances** exploration and exploitation
- Common refinement: **Decay** ϵ over time

Visual Decision Process:



Exam Trap: ϵ -greedy with decay is usually the **best practical** strategy!

How Q-Values Are Calculated in MAB

Q-Value = Average of Observed Rewards:

$$Q_k^n = \frac{1}{n} \sum_{j=1}^n R_j$$

where n = number of times machine k has been played

Example: Machine A played 5 times:

- Rewards: [1.2, 0.8, 1.5, 0.3, 1.1]
- Sum = $1.2 + 0.8 + 1.5 + 0.3 + 1.1 = 4.9$
- $Q(A) = 4.9 / 5 = 0.98$

Incremental Update Formula:

$$Q_k^n = Q_k^{n-1} + \frac{1}{n}(R_n - Q_k^{n-1})$$

This is computationally efficient - just update the average with new data!

Step-by-Step Example:

- 1 Initial: $Q = 0, n = 0$
- 2 Play 1: $R = 1.2, Q = 0 + (1/1)(1.2 - 0) = 1.2$
- 3 Play 2: $R = 0.8, Q = 1.2 + (1/2)(0.8 - 1.2) = 1.0$
- 4 Play 3: $R = 1.5, Q = 1.0 + (1/3)(1.5 - 1.0) = 1.167$

Exam Trap: Q-values are [averages](#), not sums or maximums! Update using the incremental formula.

MAB - ϵ Decay Calculation

How ϵ Decays Over Time

Exponential Decay Formula:

$$\epsilon_t = \beta^{t-1}$$

where:

- β = decay factor ($0 < \beta < 1$)
- t = trial number

Example with $\beta = 0.85$:

Trial (t)	$\epsilon_t = 0.85^{t-1}$	Decision
1	$0.85^0 = 1.00$	Always Explore
2	$0.85^1 = 0.85$	85% Explore, 15% Exploit
5	$0.85^4 = 0.522$	52.2% Explore, 47.8% Exploit
10	$0.85^9 = 0.232$	23.2% Explore, 76.8% Exploit
20	$0.85^{19} = 0.046$	4.6% Explore, 95.4% Exploit
50	$0.85^{49} = 0.0004$	Almost Pure Exploit

Key Insight:

- High ϵ early = **More exploration**
- Low ϵ later = **More exploitation**
- When ϵ approaches 0, the agent is **fully exploiting**

Exam Trap: ϵ starts at 1 (trial 1) and decays to 0 - formula is $\epsilon_t = \beta^{t-1}$, not β^t !

MAB - Worked Example

MAB - Step-by-Step Worked Example

Setup:

- 3 slot machines: A, B, C
- True means: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- Decay factor: $\beta = 0.85$
- Rewards drawn from $\text{Normal}(\mu_i, 1)$

Trial 1: $\epsilon = 1.00 \rightarrow$ Random number $0.7 < 1.0 \rightarrow$ Explore

- Choose Machine A, Reward = 1.2
- $Q(A) = 1.2$, $Q(B) = 0$, $Q(C) = 0$

Trial 2: $\epsilon = 0.85 \rightarrow$ Random number $0.99 \geq 0.85 \rightarrow$ Exploit

- Best machine: A ($Q=1.2$)
- Play A, Reward = 0.8
- $Q(A) = (1.2 + 0.8)/2 = 1.0$

Trial 3: $\epsilon = 0.7225 \rightarrow$ Random number $0.65 < 0.7225 \rightarrow$ Explore

- Choose Machine B, Reward = 1.7
- $Q(B) = 1.7$

Continue for many trials...

After 10 Trials:

Machine	Q-Value
A	0.80
B	0.50
C	0.70

Exam Trap: The estimated Q-values may differ from true means due to **sampling error** - more trials = more accurate estimates!

MAB - Greedy Strategy Example

Greedy Strategy - The Risk of Pure Exploitation

Scenario:

- 3 machines with Q-values after 5 plays:
 - $Q(A) = 1.5$ (from 5 lucky plays)
 - $Q(B) = 0.0$ (never played)
 - $Q(C) = 0.0$ (never played)
- True means: $\mu_A = 0.2$, $\mu_B = 2.0$, $\mu_C = 1.8$

Greedy Strategy:

- Always plays Machine A (highest Q-value = 1.5)
- Expected reward per play = 0.2
- Total reward = $0.2 \times 1000 = 200$

Optimal Strategy (if known):

- Always plays Machine B (true mean = 2.0)
- Expected reward per play = 2.0
- Total reward = $2.0 \times 1000 = 2000$

The Problem:

- Greedy got **stuck** with a suboptimal machine
- **Exploration** would have discovered the better machine
- This is why we need ϵ -greedy!

Regret:

- $\text{Regret} = \text{Reward}_{\text{optimal}} - \text{Reward}_{\text{actual}}$
- $= 2000 - 200 = 1800$ (huge loss!)

Exam Trap: Greedy strategy can be **catastrophically suboptimal** if the initially best machine is **not** the true best!

MAB - Random Strategy Example

Random Strategy - The Problem of Pure Exploration

Scenario:

- 3 machines with true means: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- 1000 plays

Random Strategy:

- Always picks a machine **uniformly at random**
- Expected reward per play = average of all means
- $= (0.8 + 0.5 + 0.7) / 3 = 0.667$
- Total reward $= 0.667 \times 1000 = 667$

Optimal Strategy (if known):

- Always plays Machine A (highest mean $= 0.8$)
- Total reward $= 0.8 \times 1000 = 800$

Comparison:

- Greedy (if lucky): 800
- Random: 667 (about 83% of optimal)
- With ϵ -greedy: typically 700-780 (near optimal)

The Problem:

- Random strategy **wastes** knowledge
- After 1000 plays, the agent knows which machine is best
- But continues to play random machines!

Exam Trap: Random strategy **never exploits** - it ignores all information gained!

MAB - ϵ -Greedy Example

ϵ -Greedy - The Balanced Approach

Setup:

- 3 machines, true means: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- $\epsilon = 0.10$ (constant)
- 1000 plays

How It Works:

- With probability 0.10: **Explore** - pick random machine
- With probability 0.90: **Exploit** - pick best machine so far

Expected Behavior:

- About 90% of plays: best machine (A) $\rightarrow$ reward 0.8
- About 10% of plays: random machines $\rightarrow$ average reward 0.667
- Expected reward per play $\approx 0.9(0.8) + 0.1(0.667) = 0.787$
- Total reward ≈ 787 (out of possible 800)

Comparison Table:

Strategy	Expected Reward	Regret
Optimal	800	0
ϵ -greedy (0.1)	787	13
Greedy (if lucky)	800	0
Greedy (if unlucky)	200	600
Random	667	133

With Decay (ϵ decreases over time):

- Early: More exploration to find best machine
- Late: More exploitation to maximize reward
- Often **outperforms** constant ϵ

Exam Trap: ϵ -greedy with decay is usually **better** than fixed ϵ !

MAB - ϵ Decay Factor Analysis

Choosing the Decay Factor β

Effect of Different β Values:

Trial (t)	$\beta = 0.85$	$\beta = 0.95$	$\beta = 0.99$
1	1.00	1.00	1.00
5	0.522	0.815	0.961
10	0.232	0.630	0.914
20	0.046	0.377	0.827
50	0.0004	0.077	0.606
100	≈ 0	0.006	0.366
200	≈ 0	3×10^{-5}	0.134

Interpretation:

- $\beta = 0.85$: **Fast decay** - explores early, exploits quickly
 - Good for simple problems where best action is found quickly
- $\beta = 0.95$: **Moderate decay** - balanced approach
 - Good for most practical problems
- $\beta = 0.99$: **Slow decay** - explores for a long time
 - Good for complex problems where best action is hard to find

When ϵ Becomes Very Small:

- After 57 trials with $\beta = 0.85$: $\epsilon \approx 0$
- After 230 trials with $\beta = 0.95$: $\epsilon \approx 0$
- After 2300 trials with $\beta = 0.99$: $\epsilon \approx 0$

Exam Trap: Smaller β = faster decay = **less exploration later!**

MAB - Thompson Sampling

Thompson Sampling - A Bayesian Alternative

The Concept:

- Instead of fixed ϵ , maintain a **probability distribution** for each machine
- Sample from each distribution and choose the **maximum sampled value**
- Naturally balances exploration and exploitation

How It Works:

- 1 Each machine has a Beta distribution (for binary rewards) or Normal distribution (for continuous)
- 2 Parameters update with each observation (Bayesian updating)
- 3 Each trial:
 - 1 Sample a value from each machine's distribution
 - 2 Choose the machine with the highest sampled value
 - 3 Observe reward and update distribution

Example - Binary Rewards (Win/Lose):

- Machine A: $\text{Beta}(5, 2) \rightarrow \text{mean} = 5/7 = 0.714$
- Machine B: $\text{Beta}(2, 3) \rightarrow \text{mean} = 2/5 = 0.400$
- Machine C: $\text{Beta}(3, 2) \rightarrow \text{mean} = 3/5 = 0.600$
- Sample: $A=0.72, B=0.35, C=0.68 \rightarrow \text{Choose A}$

Advantages over ϵ -Greedy:

- No ϵ parameter to tune
- Naturally balances exploration based on uncertainty
- Machines with high uncertainty get explored more
- Theoretically optimal (Bayesian)

Disadvantages:

- Requires assumptions about reward distribution
- More computationally expensive

Exam Trap: Thompson Sampling is **Bayesian** - it maintains uncertainty and samples from it!

MAB - Regret Analysis

Regret - Measuring Performance in MAB

Definition:

$$\text{Regret}(T) = T \times \mu^* - \sum_{t=1}^T \mu_{a_t}$$

where:

- T = total number of trials
- μ^* = true mean of the best machine
- μ_{a_t} = true mean of the machine played at time t

Interpretation:

- Regret is the **opportunity cost** of not knowing the best machine
- Cumulative loss from suboptimal choices
- Goal of RL: **Minimize regret**

Example:

- True means: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- $\mu^* = 0.8$ (Machine A)
- If agent plays Machine B 100 times:
 - Regret per play = $0.8 - 0.5 = 0.3$
 - Total regret = $0.3 \times 100 = 30$

Expected Regret for Different Strategies:

Strategy	Expected Regret (T=1000)
Optimal	0
ϵ -greedy (0.1)	13
Greedy (unlucky)	600
Random	133

Key Insight:

- Good algorithms achieve **sublinear regret** - grows slower than T
- ϵ -greedy with decay achieves $O(\log T)$ regret
- This means average regret per trial $\rightarrow 0$ as $T \rightarrow \infty$

Exam Trap: Regret is **cumulative loss** from suboptimal choices - minimize it!

MAB - Real-World Application: A/B Testing

A/B Testing as a Multi-Armed Bandit Problem

Traditional A/B Testing:

- Split traffic 50/50 between options
- Test for statistical significance
- Then implement the winner
- Problem: **Wastes traffic** on inferior options

MAB Approach to A/B Testing:

- Treat each variant as a **slot machine**
- Use ϵ -greedy or Thompson Sampling
- Dynamically allocate more traffic to better-performing variants
- Gradually learn which variant is best

Example - Ad Click-Through Rates:

- Variant A: 5% CTR (true)
- Variant B: 3% CTR (true)
- Variant C: 4% CTR (true)

Traditional A/B Test (100,000 impressions):

- Each variant gets 33,333 impressions
- Expected clicks = $0.05(33,333) + 0.03(33,333) + 0.04(33,333) = 4,000$

MAB Approach (100,000 impressions):

- Early: Explore all variants
- Later: Allocate 90% to best variant (A)
- Expected clicks $\approx 0.9(0.05)(100,000) + 0.05(0.03)(100,000) + 0.05(0.04)(100,000)$
- $\approx 4,500 + 150 + 200 = 4,850$
- **21% more clicks than A/B test!**

Exam Trap: MAB can significantly **outperform** traditional A/B testing by reducing regret!

MAB - Summary and Key Takeaways

MAB - Complete Summary

Key Concepts to Remember:

1 MAB Characteristics:

- Single state (environment doesn't change)
- Actions give immediate rewards
- No long-term consequences

2 Three Strategies:

- 1 Greedy = Pure exploitation (risks missing better options)
- 2 Random = Pure exploration (ignores learned knowledge)
- 3 ϵ -Greedy = Balanced approach (best practical choice)

3 Q-Value Update:

- $Q_k^n = \frac{1}{n} \sum R_j$ (average of observed rewards)
- Incremental: $Q_k^n = Q_k^{n-1} + \frac{1}{n}(R_n - Q_k^{n-1})$

4 ϵ Decay:

- $\epsilon_t = \beta^{t-1}$
- Smaller β = faster decay = less exploration later

5 Regret:

- Loss from not knowing the optimal action
- Goal: Minimize cumulative regret

Exam Success: MAB is the foundation of RL - master it and the rest follows!

Monte Carlo Methods - Learning from Complete Episodes

What is Monte Carlo RL?

- Learns from **complete episodes**
- Episode = full sequence from start to terminal state
- Example: Complete game of chess or Nim

Key Characteristic:

- **Wait until episode ends** to update
- Uses **actual returns** (G_t), not estimates
- **Unbiased** - learns from real experience

MC vs MAB:

- MAB: Each trial is independent, one-step
- MC: Multiple steps per episode, actions affect future states

When to Use MC:

- Tasks with **defined terminal states** (episodic)
- When you want **unbiased** estimates
- When episodes are not too long

Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

where G_t is the total return from time t to episode end

Exam Trap: MC requires **complete episodes** - cannot be used for continuous tasks!

Monte Carlo - Episode Definition

What is an Episode in Monte Carlo?

Definition:

- A **complete trajectory** from start to terminal state
- Sequence: $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T$
- Terminal state S_T ends the episode

Examples:

- 1 **Game of Nim**: Start with 6 coins $\rightarrow$ coin removal $\rightarrow$ last coin wins
- 2 **Chess**: Opening move $\rightarrow$... $\rightarrow$ Checkmate
- 3 **Board Game**: Start $\rightarrow$ moves $\rightarrow$ Win/Lose

Return Calculation:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$

where T is the terminal time step

Example - Nim with 6 coins:

- Episode length: 2 steps (if Player 1 wins)
- Step 1: Pick 3 coins ($S=6 \rightarrow S=3$)
- Step 2: Opponent picks 2 coins ($S=3 \rightarrow S=1$)
- Step 3: Player 1 picks 1 coin ($S=1 \rightarrow S=0$, Win!)
- Terminal state: $S=0$ (no coins left)

Update Only at Episode End:

- No updates happen within the episode
- All state-action pairs encountered are updated at episode end

Exam Trap: MC updates **only at episode end** - no online learning within episodes!

Monte Carlo - Return Calculation

Calculating Returns in Monte Carlo

The Return G_t Formula:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

Example - Nim with $\gamma = 1$:

- Episode: $S=6 \rightarrow S=3 \rightarrow S=1 \rightarrow S=0$ (Win)
- Rewards: $R_1 = 0, R_2 = 0, R_3 = 1$
- For step 1 ($t=0$): $G_0 = 0 + 0 + 1 = 1$
- For step 2 ($t=1$): $G_1 = 0 + 1 = 1$
- For step 3 ($t=2$): $G_2 = 1$

Example - Nim with $\gamma = 0.9$:

- $G_0 = R_1 + 0.9R_2 + 0.9^2R_3 = 0 + 0 + 0.81 = 0.81$
- $G_1 = R_2 + 0.9R_3 = 0 + 0.9 = 0.9$
- $G_2 = R_3 = 1$

Why Discount Future Rewards?

- Future rewards are **less certain**
- Discounting gives more weight to **immediate rewards**
- γ close to 0: Short-term focus
- γ close to 1: Long-term focus

MC Update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

The target is the **actual return** G_t

Exam Trap: MC uses G_t (actual return) as the target, not estimated future values!

Monte Carlo - First-Visit vs Every-Visit

First-Visit vs Every-Visit MC

The Distinction:

- In an episode, the same state-action pair (s, a) may be visited multiple times
- Two ways to handle this:
 - 1 **First-Visit MC**: Only update the first time (s, a) is encountered
 - 2 **Every-Visit MC**: Update every time (s, a) is encountered

First-Visit MC:

- Track which (s, a) have been seen in the episode
- Only update $Q(s,a)$ the **first time** it occurs
- Uses the return from that first occurrence
- **Unbiased** estimate of $Q(s,a)$

Every-Visit MC:

- Update $Q(s,a)$ **every time** it occurs
- Uses the return from each occurrence
- **Slightly biased** but often lower variance

Example:

- Episode: $(S1,A1)$, $(S2,A2)$, $(S1,A1)$, $(S3,A3)$, Terminal
- First-visit: Update $(S1,A1)$ only once (using first occurrence)
- Every-visit: Update $(S1,A1)$ twice (both occurrences)

When to Use Each:

- First-visit: Theoretical purity, unbiased estimates
- Every-visit: More sample-efficient, practical use

Exam Trap: First-visit MC is **unbiased**; Every-visit MC has **lower variance** but slight bias!

Monte Carlo - NIM Example Step-by-Step

MC Applied to NIM - Step-by-Step Example

Setup:

- 6 coins, remove 1, 2, or 3 coins
- $\alpha = 0.05$, $\gamma = 1$
- Opponent plays randomly (not optimally)

Episode 1:

- 1 $S=6$, $A=3 \rightarrow S'=3$ (opponent picks 2)
- 2 $S=1$, $A=1 \rightarrow S'=0$ (Player 1 wins!)
- 3 Return: $G = 1$ (win)

MC Updates (Episode End):

- For (6,3): $Q(6, 3) = 0 + 0.05(1 - 0) = 0.05$
- For (1,1): $Q(1, 1) = 0 + 0.05(1 - 0) = 0.05$

Episode 2:

- 1 $S=6$, $A=1 \rightarrow S'=5$ (opponent picks 1)
- 2 $S=4$, $A=3 \rightarrow S'=1$ (opponent picks 1)
- 3 $S=0$ (Player 1 loses!)
- 4 Return: $G = -1$ (loss)

MC Updates (Episode End):

- For (6,1): $Q(6, 1) = 0 + 0.05(-1 - 0) = -0.05$
- For (4,3): $Q(4, 3) = 0 + 0.05(-1 - 0) = -0.05$

After Many Episodes:

- Q-values converge to optimal values
- Optimal strategy: 6 coins $\rightarrow$ pick 2

Exam Trap: MC updates **all state-action pairs** in the episode using the same G value!

MC NIM - Q-Table Evolution Over Episodes

After Episode 1:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0	0	0.05

After Episode 2:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	-0.05
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	-0.05	0	0.05

After Episode 3:

- $Q(6,3): 0.05 + 0.05(1 - 0.05) = 0.0975$
- $Q(1,1): 0.05 + 0.05(1 - 0.05) = 0.0975$

After 1,000,000 Episodes:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	1	-1	-0.071	0.380	0	0.774
Pick 2	0	1	-1	-0.110	0	1
Pick 3	0	0	1	-0.858	0	0.020

Optimal Strategy:

- S=6: Pick 2 ($Q=1$)
- S=4: Pick 1 ($Q=0.380$)
- S=3: Pick 3 ($Q=1$)
- S=2: Pick 2 ($Q=1$)
- S=1: Pick 1 ($Q=1$)

Exam Trap: Q-values converge to 1 for winning actions, -1 for losing actions!

Monte Carlo - Advantages and Disadvantages

MC - Pros and Cons

Advantages:

- 1 Unbiased estimates - uses actual returns, not estimates
- 2 Simple to understand and implement
- 3 Works well with long episodes (uses all information)
- 4 No bootstrapping - no risk of propagating errors

Disadvantages:

- 1 High variance - returns can fluctuate significantly
- 2 Requires complete episodes - cannot handle continuous tasks
- 3 Slow learning - must wait until episode end to update
- 4 Inefficient for long episodes - updates are infrequent

When to Use MC:

- Tasks with clear terminal states (episodic)
- When you can run many episodes
- When you want unbiased value estimates
- When episode lengths are reasonable

Comparison to TD:

MC	TD
Unbiased	Biased (bootstrapping)
High variance	Low variance
Episode-end updates	Step-by-step updates
Episodic tasks only	Episodic and continuous tasks

Exam Trap: MC = unbiased but high variance; TD = biased but low variance!

Monte Carlo - Complete Summary

Key Concepts to Remember:

1 Definition:

- Learns from **complete episodes**
- Updates only at episode end
- Uses actual returns (G_t)

2 Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

3 Return Calculation:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

4 First-Visit vs Every-Visit:

- First-visit: Update only first occurrence (unbiased)
- Every-visit: Update all occurrences (lower variance)

5 Requirements:

- **Must have** terminal states (episodic tasks)
- Cannot handle continuous tasks

6 Trade-off:

- **Unbiased** but **high variance**

Exam Success: Remember MC's key limitation: **episodic tasks only!**

Temporal Difference - Learning Step by Step

What is TD Learning?

- Learns from **incomplete episodes**
- Updates **after each step** (online learning)
- Uses **bootstrapping** - estimates of future values

Key Characteristic:

- **Doesn't wait** for episode end
- Updates using $R_{t+1} + \gamma V(s')$
- **Biased** but **lower variance**

TD vs MC:

MC	TD
Episode-end updates	Step-by-step updates
Uses G_t (actual return)	Uses $R_{t+1} + \gamma V(s')$ (estimate)
Unbiased	Biased (bootstrapping)
High variance	Low variance
Episodic only	Episodic and continuous

TD Update Formula (Value Function):

$$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$$

TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

This is the "**surprise**" - the difference between new information and current estimate

Exam Trap: TD uses **bootstrapping** - it updates using estimates of future values, not actual returns!

TD - Bootstrapping Explained

Bootstrapping - The Core of TD Learning

What is Bootstrapping?

- Using a **current estimate** to update another estimate
- "Pulling yourself up by your own bootstraps"
- Instead of waiting for the final outcome, use your current best guess

MC vs TD Bootstrapping:

- **MC**: No bootstrapping - uses actual returns
- **TD**: Bootstrapping - uses estimates of future values

Example - Nim:

- $S=6$, take action that leads to $S'=1$
- MC: Wait until episode end to know G
- TD: Use $V(1)$ (current estimate) plus immediate reward

TD Target:

$$R_{t+1} + \gamma V(s')$$

- This is a **better guess** than $V(s)$ alone
- It uses immediate reward plus estimate of future

Why Bootstrapping Works:

- Updates can be made **immediately**
- Doesn't require complete episodes
- Often **converges faster** than MC

TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

- Positive δ : New information suggests state is better than thought
- Negative δ : New information suggests state is worse than thought
- Zero δ : New information matches current estimate

Exam Trap: Bootstrapping introduces **bias** but **reduces variance!**

Q-Learning - The Most Popular TD Algorithm

Q-Learning Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Key Features:

- 1 **Off-policy:** Learns optimal policy regardless of behavior policy
- 2 **Model-free:** No environment model needed
- 3 **TD method:** Uses bootstrapping

Why "Off-Policy"?

- The update uses $\max_{a'} Q(s', a')$ (the best possible action)
- It doesn't matter which action was actually taken
- The agent is learning about the **optimal policy**, not the current policy

Algorithm Steps:

- 1 Initialize $Q(s,a)$ arbitrarily
- 2 For each episode:
 - 1 Initialize state s
 - 2 For each step:
 - 1 Choose a from s using ϵ -greedy (behavior policy)
 - 2 Take action a , observe R and s'
 - 3 Update $Q(s,a)$ using TD target
 - 4 $s \leftarrow s'$

Example - Simple Grid World:

- State: Position (x,y)
- Actions: Up, Down, Left, Right
- Reward: +1 at goal, -1 for walls
- Q-learning learns optimal path to goal

Exam Trap: Q-learning is **off-policy** - learns optimal policy while following any policy (usually ϵ -greedy)!

SARSA - On-Policy TD Learning

SARSA Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma Q(s', a') - Q(s, a))$$

where a' is the **actual action** taken in state s'

Key Features:

- ① **On-policy:** Learns the policy being followed
- ② **Model-free:** No environment model needed
- ③ **TD method:** Uses bootstrapping

Why "On-Policy"?

- The update uses $Q(s', a')$ where a' is the **actual** next action
- The agent learns about the **current policy**, not the optimal policy
- Policy and learning are linked

SARSA vs Q-Learning:

Q-Learning	SARSA
Off-policy	On-policy
Uses $\max_{a'} Q(s', a')$	Uses $Q(s', a')$ (actual action)
More aggressive	More conservative
Learns optimal policy	Learns current policy

SARSA Algorithm Steps:

- ① Initialize $Q(s, a)$ arbitrarily
- ② For each episode:
 - ① Initialize state s
 - ② Choose a from s using policy (e.g., ϵ -greedy)
 - ③ For each step:
 - ① Take action a , observe R and s'
 - ② Choose a' from s' using policy
 - ③ Update $Q(s, a)$ using TD target with a'
 - ④ $s \leftarrow s', a \leftarrow a'$

Exam Trap: SARSA is **on-policy**; Q-learning is **off-policy**. SARSA is more conservative!

TD - Q-Learning vs SARSA Comparison

Q-Learning vs SARSA - Detailed Comparison

Key Difference:

- Q-learning: $Q(s, a) \leftarrow Q(s, a) + \alpha(R + \gamma \max_{a'} Q(s', a') - Q(s, a))$
- SARSA: $Q(s, a) \leftarrow Q(s, a) + \alpha(R + \gamma Q(s', a') - Q(s, a))$

Behavioral Differences:

Q-Learning	SARSA
Considers the best possible next action	Considers the actual next action
More aggressive - learns risky policies	More conservative - learns safe policies
May choose actions that lead to danger if the danger is avoided optimally	May avoid actions that could lead to danger under current policy
Learns the optimal policy	Learns the current policy

Example - Cliff Walking:

- Agent must walk along a cliff to reach the goal
- Falling off cliff gives large negative reward

Q-Learning:

- Learns the optimal path (right next to the cliff)
- This path is risky if the agent sometimes makes mistakes

SARSA:

- Learns a safer path (further from the cliff)
- Accounts for the fact that the agent may take suboptimal actions

Which to Use?

- Q-learning: When you want optimal policy, can explore safely
- SARSA: When safety is important, exploration is risky

Exam Trap: Q-learning is more **aggressive**; SARSA is more **conservative**!

TD - NIM Example Step-by-Step

TD Applied to NIM - Step-by-Step Example

Setup:

- 6 coins, remove 1, 2, or 3 coins
- $\alpha = 0.05$, $\gamma = 1$
- Opponent plays randomly

Episode 1:

- 1 $S=6$, $A=3 \rightarrow S'=3$ (opponent picks 2)
- 2 $S=1$, $A=1 \rightarrow S'=0$ (Player 1 wins!)

TD Updates (Step by Step):

- **Step 1:** $S=6$, $A=3$, $S'=3$, $R=0$
 - $V(3) = 0$ (current estimate, all 0)
 - TD target $= 0 + 1(0) = 0$
 - $Q(6,3) = 0 + 0.05(0 - 0) = 0$
- **Step 2:** $S=1$, $A=1$, $S'=0$, $R=1$
 - $V(0) = 0$ (terminal state value)
 - TD target $= 1 + 1(0) = 1$
 - $Q(1,1) = 0 + 0.05(1 - 0) = 0.05$
- Update $V(1)$: $V(1) = \max Q(1,a) = 0.05$

Episode 2:

- 1 $S=6$, $A=1 \rightarrow S'=5$ (opponent picks 1)
- 2 $S=4$, $A=3 \rightarrow S'=1$ (opponent picks 1)
- 3 $S=0$ (Player 1 loses!)

TD Updates:

- **Step 1:** $S=6$, $A=1$, $S'=5$, $R=0$
 - $V(5) = 0$
 - $Q(6,1) = 0 + 0.05(0 - 0) = 0$
- **Step 2:** $S=4$, $A=3$, $S'=1$, $R=0$
 - $V(1) = 0.05$ (from Episode 1)
 - TD target $= 0 + 1(0.05) = 0.05$
 - $Q(4,3) = 0 + 0.05(0.05 - 0) = 0.0025$

TD NIM - Q-Table Evolution Over Episodes

After Episode 1:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0	0	0

$V(1) = 0.05$

After Episode 2:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0.0025	0	0

$V(1) = 0.05, V(4) = 0.0025$

After Episode 3:

① $S=6, A=3, S'=1, R=0$

- $V(1) = 0.05$
- $Q(6,3) = 0 + 0.05(0.05 - 0) = 0.0025$

② $S=1, A=1, S'=0, R=1$

- $Q(1,1) = 0.05 + 0.05(1 - 0.05) = 0.0975$

After 1,000,000 Episodes:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	1	-1	-0.105	0.383	0	0.845
Pick 2	0	1	-1	0.058	0	1
Pick 3	0	0	1	-0.842	0	0.011

Optimal Strategy: Same as MC - pick 2 when 6 coins remain!

Exam Trap: Both MC and TD converge to the **same optimal strategy!**

TD - Advantages and Disadvantages

TD - Pros and Cons

Advantages:

- 1 Can learn online - updates after each step
- 2 Works for continuous tasks - no terminal state needed
- 3 Lower variance than MC due to bootstrapping
- 4 More sample efficient - uses information immediately
- 5 Often converges faster than MC

Disadvantages:

- 1 Biased - uses estimates of future values
- 2 May propagate errors - initial bad estimates persist
- 3 Requires careful tuning of α and γ
- 4 Can be unstable with function approximation

When to Use TD:

- Continuous (non-episodic) tasks
- When you need online learning
- When episodes are very long
- When you want faster learning

Comparison Table:

MC	TD
Unbiased	Biased
High variance	Low variance
Episode-end updates	Step-by-step updates
Episodic tasks only	Episodic and continuous
Slower learning	Faster learning

Exam Trap: TD is more flexible but has bias; MC is unbiased but less flexible!

TD - TD(λ) - The Unified View

TD(λ) - Bridging MC and TD

The Idea:

- MC = TD(1) - uses full return
- TD(0) = One-step TD - uses immediate reward + one lookahead
- TD(λ) - combines many n-step returns

n-Step Returns:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(s_{t+n})$$

- n=1: TD(0) target
- n= ∞ : MC target (when episode ends)

λ -Return:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- $\lambda = 0$: TD(0) (one-step)
- $\lambda = 1$: MC (full return)
- $0 < \lambda < 1$: Mix of n-step returns

Intuition:

- λ controls how far into the future you look
- Lower λ : More like TD (lower variance, higher bias)
- Higher λ : More like MC (higher variance, lower bias)

TD(λ) Update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (G_t^\lambda - Q(s, a))$$

Advantages:

- Can be **more efficient** than TD(0)
- Can be **more accurate** than MC
- Provides a **unified framework**

Exam Trap: TD(λ) generalizes both MC and TD - $\lambda = 0 = \text{TD}$, $\lambda = 1 = \text{MC}$!

TD - Summary and Key Takeaways

TD - Complete Summary

Key Concepts to Remember:

1 TD Learning:

- Learns **online** - updates after each step
- Uses **bootstrapping** - estimates future values

2 TD Update:

$$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$$

3 TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

4 Q-Learning:

- Off-policy, uses $\max_{a'} Q(s', a')$
- More aggressive

5 SARSA:

- On-policy, uses $Q(s', a')$ (actual action)
- More conservative

6 TD(λ):

- $\lambda = 0 = \text{TD}$, $\lambda = 1 = \text{MC}$
- Can be more efficient than TD(0)

7 TD vs MC:

- TD: Biased, lower variance, online
- MC: Unbiased, higher variance, episode-end

Exam Success: Understand the **bias-variance trade-off** between MC and TD!

Deep Reinforcement Learning - Scaling RL

The Problem with Tabular Methods:

- Q-table stores $Q(s,a)$ for every state-action pair
- Number of states can be **enormous** (or infinite)
- Example: Atari games have 10^{50} possible states!
- Self-driving cars have continuous state spaces

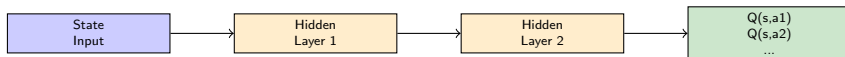
The Solution: Deep RL

- Use **neural networks** to approximate $Q(s,a)$
- $Q(s, a) \approx Q(s, a; \theta)$ where θ = network weights
- Neural network takes state as input, outputs Q-values for all actions

Why Neural Networks?

- 1 **Generalization**: Similar states get similar Q-values
- 2 **Compact representation**: No need to store all states
- 3 **Continuous spaces**: Can handle continuous state inputs

Deep RL Architecture:



Exam Trap: Deep RL is needed when state spaces are **too large for tables**!

DQN - Introduction

Deep Q-Network - The Breakthrough Algorithm

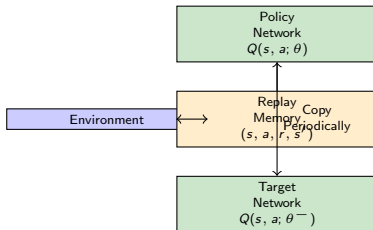
What is DQN?

- Combines Q-learning with [deep neural networks](#)
- Developed by DeepMind (2015)
- Achieved human-level performance on Atari games

Three Key Innovations:

- 1 [Experience Replay](#): Store experiences and sample randomly
- 2 [Target Network](#): Separate network for stable targets
- 3 [\$\epsilon\$ -Greedy](#): For exploration

DQN Architecture:



How DQN Works:

- 1 Interact with environment and store experiences
- 2 Sample a random mini-batch from replay memory
- 3 Update the policy network using the TD target from the target network
- 4 Periodically copy policy network weights to the target network

Exam Trap: DQN's success comes from [experience replay](#) and [target network](#).

DQN - Experience Replay

Experience Replay - Why It's Critical

The Problem:

- Consecutive experiences in RL are **highly correlated**
- Example: In a game, consecutive frames are nearly identical
- Correlated data breaks the i.i.d. assumption of supervised learning

The Solution: Experience Replay

- 1 Store experiences (s, a, r, s') in a **replay memory**
- 2 During training, sample **random mini-batches** from memory
- 3 This breaks temporal correlation

Why Random Sampling?

- Reduces correlation between samples
- Makes learning more stable
- Reuses experiences multiple times (more efficient)

Replay Memory Structure:

- Fixed size (e.g., 1,000,000 experiences)
- Oldest experiences are discarded (FIFO)
- Random sampling (uniform or prioritized)

Benefits:

- 1 **Stable learning**: Breaks correlations
- 2 **Efficient**: Reuses experiences
- 3 **Reduces variance**: Averaging over random samples

Prioritized Experience Replay:

- Sample experiences with **higher TD error** more often
- Focuses on experiences that are most "surprising"
- Can improve learning efficiency

Exam Trap: Experience replay breaks **temporal correlation** - this is key to DQN's stability!

DQN - Target Network

Target Network - Stabilizing the Moving Target

The Problem:

- Q-learning uses $R + \gamma \max_{a'} Q(s', a')$ as target
- But $Q(s', a')$ is the **same network** being updated!
- This creates a **"chasing the moving target"** problem

The Solution: Target Network

- Use a **separate network** for the target
- Target network weights: θ^-
- Policy network weights: θ

How It Works:

- 1 Policy network: Used to select actions (ϵ -greedy)
- 2 Target network: Used to compute TD target
- 3 Update policy network: $L = (Q(s, a; \theta) - (R + \gamma \max_{a'} Q(s', a'; \theta^-)))^2$
- 4 Periodically copy: $\theta^- \leftarrow \theta$

Update Frequency:

- Typical: Copy every **500** steps
- Too frequent: Target moves too fast (unstable)
- Too infrequent: Target is outdated (slow learning)

Benefits:

- 1 **Stable targets**: Target doesn't change every step
- 2 **More stable learning**: Reduces oscillations
- 3 **Convergence**: Helps converge to optimal policy

Example:

- Policy network: Updated every step
- Target network: Updated every 1000 steps
- TD target uses target network, not policy network

Exam Trap: Target network is updated **periodically**, not every step - this stabilizes learning!

DQN - Loss Function

DQN Loss Function and Training

Loss Function (MSE):

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} [(Q(s, a; \theta) - (r + \gamma \max_{a'} Q(s', a'; \theta^-)))^2]$$

Components:

- $Q(s, a; \theta)$: Policy network prediction
- $r + \gamma \max_{a'} Q(s', a'; \theta^-)$: TD target from target network
- $U(D)$: Uniform random sampling from replay memory

Training Process:

- 1 Sample random mini-batch from replay memory
- 2 For each experience:
 - 1 Compute target: $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$
 - 2 Compute prediction: $Q(s, a; \theta)$
 - 3 Compute loss: $(y - Q(s, a; \theta))^2$
- 3 Average loss over mini-batch
- 4 Backpropagate and update θ

Gradient Update:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$$

Hyperparameters:

- Learning rate α : 0.001 (typical)
- Mini-batch size: 32 or 64
- Replay memory size: 1,000,000

Why MSE?

- MSE is differentiable
- Consistent with regression framework
- Works well with gradient descent

Exam Trap: DQN loss is **MSE** between policy network and target network outputs!

DQN - Training Process

DQN Training - Step-by-Step Process

Algorithm:

1 Initialize:

- Policy network $Q(s, a; \theta)$ with random weights
- Target network $Q(s, a; \theta^-)$ with same weights
- Replay memory D of capacity N

2 For each episode:

1 Initialize state s

2 For each step:

- 1 With probability ϵ : Choose random action (explore)
- 2 Otherwise: Choose $a = \arg \max_a Q(s, a; \theta)$ (exploit)
- 3 Take action a , observe reward r and next state s'
- 4 Store (s, a, r, s') in D
- 5 Sample random mini-batch from D
- 6 Compute loss and update θ
- 7 Periodically update $\theta^- \leftarrow \theta$
- 8 Set $s \leftarrow s'$

ϵ Decay in DQN:

- Start with $\epsilon = 1.0$ (pure exploration)
- Decay to $\epsilon = 0.01$ (mostly exploitation)
- Linear decay over first 1,000,000 steps

Stopping Criteria:

- Maximum number of steps reached
- No improvement in Q-values
- Episode reward converges

Typical Values:

- $\gamma = 0.99$ (discount factor)
- $\alpha = 0.00025$ (learning rate)
- Replay memory: 1,000,000 experiences
- Mini-batch: 32
- Target update: every 10,000 steps

DQN - Convergence and Stability

DQN - Ensuring Stable Learning

Challenges in Deep RL:

- 1 **Non-stationarity**: The target distribution changes
- 2 **Correlation**: Consecutive samples are correlated
- 3 **Divergence**: Q-learning with function approximation can diverge

DQN Solutions:

- 1 **Experience Replay**: Breaks correlation
- 2 **Target Network**: Stabilizes targets
- 3 **Gradient Clipping**: Prevents exploding gradients
- 4 **Frame Stacking**: Gives temporal context

Gradient Clipping:

- Clip gradients to a maximum value
- Prevents very large updates
- Common threshold: 1.0 or 10.0

Frame Stacking (for Atari):

- Stack last 4 frames
- Provides temporal information (motion)
- Input: 84x84x4 image

Double DQN:

- Uses target network for action selection too
- Reduces overestimation bias
- $y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$

Dueling DQN:

- Separate value and advantage streams
- Better estimate of state values
- $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$

Exam Trap: DQN can be unstable - use [experience replay](#) and [target networks](#) for stability!

DQN - Advantages and Limitations

DQN - Pros, Cons, and Extensions

Advantages:

- 1 Handles large state spaces - neural networks generalize
- 2 State-of-the-art on many games
- 3 Can learn directly from raw pixels
- 4 Combines deep learning with RL

Limitations:

- 1 Computationally expensive - training can take days
- 2 Requires many samples - millions of interactions
- 3 Limited to discrete action spaces - outputs Q-values per action
- 4 Can be unstable - hyperparameter sensitive

DQN Extensions:

- 1 Double DQN: Reduces overestimation bias
- 2 Dueling DQN: Separate value and advantage streams
- 3 Prioritized Experience Replay: Sample important experiences more often
- 4 Rainbow: Combines multiple DQN improvements

When to Use DQN:

- Large discrete action spaces
- High-dimensional state spaces (images)
- When you have computational resources
- When samples are plentiful

When NOT to Use DQN:

- Small state spaces (tabular methods are better)
- Continuous action spaces (use policy-based methods)
- When samples are expensive

Exam Trap: DQN is **only for discrete action spaces** - use policy methods for continuous actions!

DQN - Complete Summary

Key Concepts to Remember:

① DQN = Q-Learning + Deep Neural Networks

- Replaces Q-table with neural network
- Input: state; Output: Q-values for all actions

② Three Key Innovations:

- ① **Experience Replay**: Random sampling breaks correlation
- ② **Target Network**: Separate network for stable targets
- ③ **ϵ -Greedy**: Exploration/exploitation balance

③ Loss Function:

$$L = \mathbb{E}[(Q(s, a; \theta) - (R + \gamma \max_{a'} Q(s', a'; \theta^-)))^2]$$

④ Update Steps:

- ① Sample mini-batch from replay memory
- ② Compute targets using target network
- ③ Update policy network
- ④ Periodically copy weights to target network

⑤ Limitations:

- Discrete actions only
- Computationally expensive
- Sample inefficient

Exam Success: Remember DQN's **three key innovations**: Experience Replay, Target Network, ϵ -Greedy!

Policy Methods - Introduction

Policy-Based Methods - Learning the Strategy Directly

Value-Based vs Policy-Based:

Value-Based	Policy-Based
Learn $Q(s,a)$	Learn $\pi(a s)$ directly
Derive policy from Q	Policy is the output
Q-learning, SARSA	Policy Gradient
Discrete actions only	Continuous actions possible

Policy Representation:

- Policy is parameterized: $\pi_{\theta}(a|s)$
- θ = parameters (neural network weights)
- Neural network takes state s as input
- Output: probability distribution over actions

Policy Gradient:

- Optimize θ to maximize expected reward
- Gradient ascent on $J(\theta) = E_{\pi}[\sum R]$
- Update: $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

Why Policy Methods?

- 1 **Continuous action spaces:** Can output continuous actions
- 2 **Stochastic policies:** Natural for partial observability
- 3 **Better convergence:** Guaranteed to converge to local optimum

Why NOT Policy Methods?

- 1 **Slow convergence:** Often slower than value-based
- 2 **Local optima:** Can get stuck in local optima
- 3 **Sample inefficient:** Often require many samples

Exam Trap: Policy methods are preferred for **continuous action spaces!**

Policy Gradient - REINFORCE

REINFORCE - Monte Carlo Policy Gradient

The REINFORCE Algorithm:

- Monte Carlo policy gradient method
- Updates after complete episodes
- Uses the actual return G_t

Update Formula:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Intuition:

- If G_t is positive (good outcome), increase probability of action
- If G_t is negative (bad outcome), decrease probability of action
- The gradient $\nabla_{\theta} \log \pi_{\theta}$ points in the direction to increase the action's probability

Algorithm Steps:

- 1 Initialize policy parameters θ
- 2 For each episode:
 - 1 Generate episode: $s_0, a_0, r_1, s_1, \dots, s_T$
 - 2 For each step t :
 - 1 Compute $G_t = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$
 - 2 Update $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$

Advantages:

- Simple to implement
- Theoretically sound
- Works for continuous actions

Disadvantages:

- High variance (like MC)
- Requires complete episodes
- Slow convergence

Baseline Subtraction (Variance Reduction):

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

where $b(s_t)$ is a baseline (often $V(s_t)$)

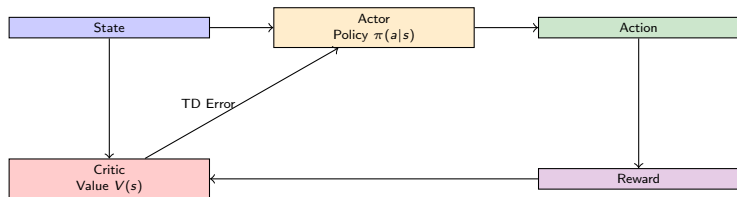
Actor-Critic Methods

Actor-Critic - The Best of Both Worlds

What is Actor-Critic?

- Combines **value-based** and **policy-based** methods
- **Actor**: Learns the policy (policy-based)
- **Critic**: Learns the value function (value-based)
- Critic evaluates actor's actions

Architecture:



How It Works:

- 1 Actor takes action based on current policy
- 2 Critic evaluates the action (TD error)
- 3 TD error: $\delta = R + \gamma V(s') - V(s)$
- 4 If $\delta > 0$: Action was better than expected $\rightarrow$ increase probability
- 5 If $\delta < 0$: Action was worse than expected $\rightarrow$ decrease probability

Advantages:

- 1 Lower variance than policy gradient
- 2 Can learn online (TD-based)
- 3 Continuous action spaces (policy-based)

Exam Trap: Actor-Critic combines **policy** (actor) and **value** (critic) learning!

Policy Methods - Summary

Policy Methods - Complete Summary

Key Concepts to Remember:

1 Policy-Based Methods:

- Learn policy $\pi_{\theta}(a|s)$ directly
- Output: action (or probability distribution)
- Better for continuous action spaces

2 Value-Based Methods:

- Learn $Q(s,a)$ or $V(s)$
- Output: value estimates
- Derive policy from values
- Better for discrete action spaces

3 REINFORCE:

- MC policy gradient
- High variance
- Requires complete episodes

4 Actor-Critic:

- Actor: Policy
- Critic: Value function
- Lower variance
- Can learn online

5 Advantages of Policy Methods:

- Continuous actions
- Stochastic policies
- Better convergence

6 Disadvantages of Policy Methods:

- Slow convergence
- Local optima
- Sample inefficient

Exam Success: Choose between value-based and policy-based based on **action space** (discrete vs. continuous)!

MDP - Formal Definition

Markov Decision Processes - The RL Framework

What is an MDP?

- Mathematical framework for modeling decision-making
- Defined by: States, Actions, Transition Probabilities, Rewards
- Assumes the [Markov property](#)

MDP Components:

- 1 **States (S)**: Set of all possible states
- 2 **Actions (A)**: Set of all possible actions
- 3 **Transition Probability**: $P(s'|s, a)$
- 4 **Reward**: $R(s, a)$ or $R(s, a, s')$
- 5 **Discount Factor**: $\gamma \in [0, 1]$

The Markov Property:

$$P(S_{t+1}|S_t, S_{t-1}, \dots, S_1) = P(S_{t+1}|S_t)$$

- Future depends only on the [current state](#)
- "Memoryless" property
- Simplifies analysis significantly

Transition Probability Matrix:

- P = matrix where $P_{ij} = P(S_{t+1} = j | S_t = i)$
- For actions: $P_{ij}^a = P(S_{t+1} = j | S_t = i, A_t = a)$
- After n steps: P^n (matrix multiplication)

Example - Simple 2-State MDP:

- States: S_1, S_2
- Actions: Left, Right
- Transition: $P(\text{Left from } S_1 \text{ to } S_2) = 0.7, P(\text{Left from } S_1 \text{ to } S_1) = 0.3$

Exam Trap: MDPs assume the [Markov property](#) - only the current state matters!

Bellman Equations - Derivation

Bellman Equations - The Recursive Foundation

The Bellman Equation (For a Fixed Policy):

$$V(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$$

Derivation:

- ① $V(s) = E[G_t | S_t = s]$
- ② $G_t = R_{t+1} + \gamma G_{t+1}$
- ③ $V(s) = E[R_{t+1} + \gamma V(S_{t+1}) | S_t = s]$
- ④ Expand expectation over actions and next states

Bellman Optimality Equation:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^*(s')]$$

For Q-Function:

$$Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a) + \gamma \max_{a'} Q(s', a')]$$

What They Tell Us:

- Value of a state = immediate reward + discounted value of next state
- This is [recursive](#)
- Can be solved using [dynamic programming](#)

Solving Bellman Equations:

- If transition probabilities known: Dynamic Programming
- If not known: MC or TD methods

Exam Trap: Bellman equations are [recursive](#) - they define $V(s)$ in terms of $V(s')$!

Bellman Equations - Examples

Bellman Equations - Worked Examples

Example 1 - Simple MDP:

- 2 states: $S1, S2$
- 2 actions: $A1, A2$ (deterministic)
- Rewards: $R(S1, A1, S2) = 10, R(S2, A2, S1) = 5$
- $\gamma = 0.9$

Bellman Equations:

- $V(S1) = 10 + \gamma V(S2)$
- $V(S2) = 5 + \gamma V(S1)$

Solve:

- $V(S1) = 10 + 0.9V(S2)$
- $V(S2) = 5 + 0.9V(S1)$
- Substitute: $V(S1) = 10 + 0.9(5 + 0.9V(S1)) = 10 + 4.5 + 0.81V(S1)$
- $0.19V(S1) = 14.5 \Rightarrow V(S1) = 76.32$
- $V(S2) = 5 + 0.9(76.32) = 73.69$

Example 2 - Grid World:

- Agent in 3x3 grid
- Goal: Reach top-right corner
- Reward: +1 at goal, -1 for walls
- $\gamma = 0.9$

Bellman Optimality:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a)[R(s, a) + \gamma V^*(s')]$$

- Solve iteratively (value iteration)
- Converges to optimal values

Exam Trap: Bellman equations can be **solved analytically** for simple MDPs!

Reinforcement Learning - Complete Summary

Core Concepts:

- Agent learns through **trial and error**
- Goal: **Maximize cumulative reward**
- Policy: Strategy for choosing actions
- Value functions: $V(s)$ and $Q(s,a)$

The Three Strategies (MAB):

- 1 Greedy (Pure Exploitation)
- 2 Random (Pure Exploration)
- 3 ϵ -Greedy (Balanced) - **Best practical choice**

Two Main Approaches:

1 Monte Carlo (MC):

- Episode-end updates
- Uses actual returns G_t
- Unbiased, high variance
- Episodic tasks only

2 Temporal Difference (TD):

- Step-by-step updates
- Uses bootstrapping
- Biased, low variance
- Episodic and continuous tasks

Two Families of Algorithms:

1 Value-Based:

- Q-learning (off-policy)
- SARSA (on-policy)
- DQN (deep learning)

2 Policy-Based:

- REINFORCE (MC)
- Actor-Critic
- Policy Gradient

Top 15 RL Exam Traps to Remember:

- 1 **Trap 1:** RL output is a **policy** (strategy), not a prediction or classification
- 2 **Trap 2:** MAB has **ONLY ONE state** - actions don't change the environment
- 3 **Trap 3:** ϵ -greedy: random number $< \epsilon \rightarrow$ **explore**; $\geq \rightarrow$ **exploit**
- 4 **Trap 4:** ϵ decays as $\epsilon_t = \beta^{t-1}$ (not β^t)
- 5 **Trap 5:** MC updates **at episode end** using G_t ; TD updates **every step** using $R_{t+1} + \gamma V(s')$
- 6 **Trap 6:** MC = **unbiased but high variance**; TD = **biased but low variance**
- 7 **Trap 7:** Q-learning is **off-policy**; SARSA is **on-policy**
- 8 **Trap 8:** α = **learning rate**; γ = **discount factor**; ϵ = **exploration rate**
- 9 **Trap 9:** DQN uses **experience replay** + **target network** for stability
- 10 **Trap 10:** Policy-based methods handle **continuous action spaces**; value-based methods handle discrete actions
- 11 **Trap 11:** MC requires **episodic tasks** (terminal states); TD handles continuous tasks
- 12 **Trap 12:** Bellman equations are **recursive** - $V(s)$ depends on $V(s')$
- 13 **Trap 13:** MDPs assume the **Markov property** - only the current state matters
- 14 **Trap 14:** Actor-Critic = **Actor** (policy) + **Critic** (value)
- 15 **Trap 15:** RLHF is **RL from human feedback** - used for fine-tuning LLMs

Remember: RL is about **learning optimal behavior through experience!** The exam tests conceptual understanding of the trade-offs between methods.